Spatial Analysis and Modeling

Spatial Clustering and Regionalization

UNIVERSITÀ
DI TORINO

Why Spatial Clustering?

Standard Clustering

- Groups similar observations
- Ignores spatial relationships
- May produce fragmented results
- Distance in feature space only

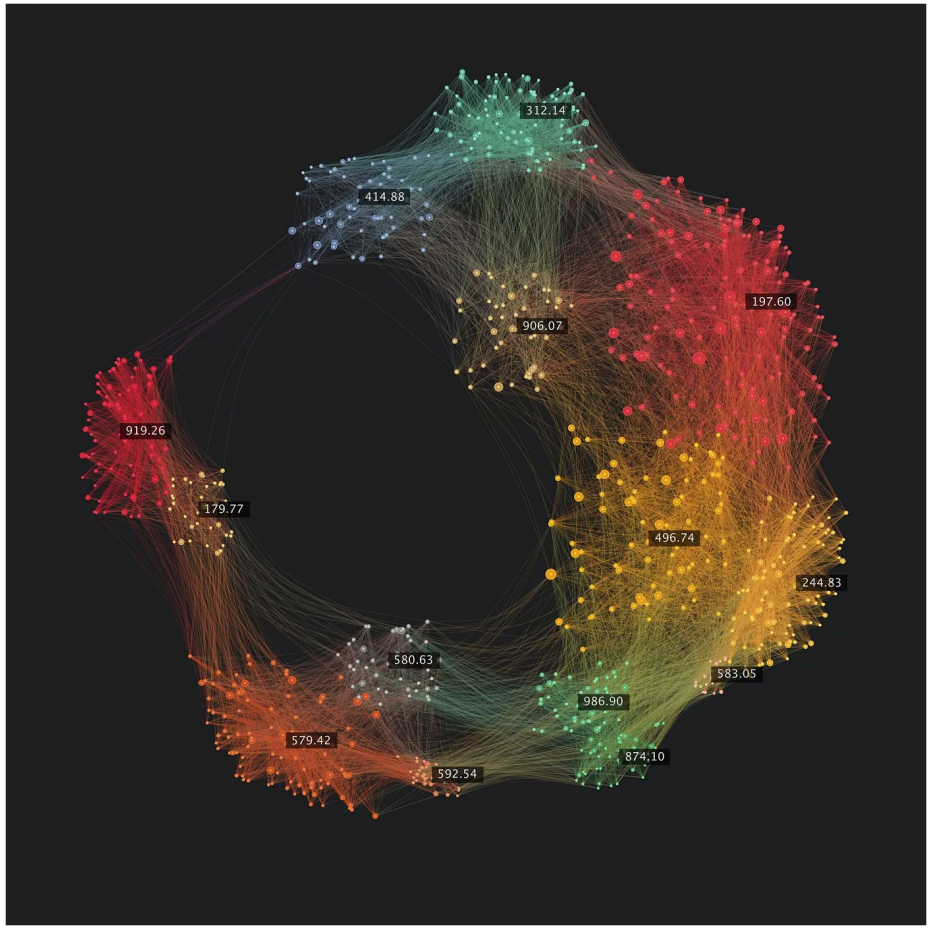
Spatial Clustering

- **Spatial autocorrelation** matters
- Geographic proximity + similarity
- Spatially coherent clusters
- Real-world geographical meaning

> **Tobler's First Law**: "Everything is related to everything else, but near things are more related than distant things"

Key Challenges in Spatial Data

- **Balancing Act**
  - **Attribute similarity** vs. **spatial proximity**
  - Risk of spatially fragmented clusters
  - Need for **geographical coherence**
- **Spatial Outliers**
  - Observations different from neighbors
  - Not global outliers, but local anomalies
  - Can distort conventional algorithms
- **Contiguity Constraints**
  - Need for connected regions
  - Applications: districts, territories, zones
  - **Regionalization** problem



Spatial Clustering

Density-Based Methods

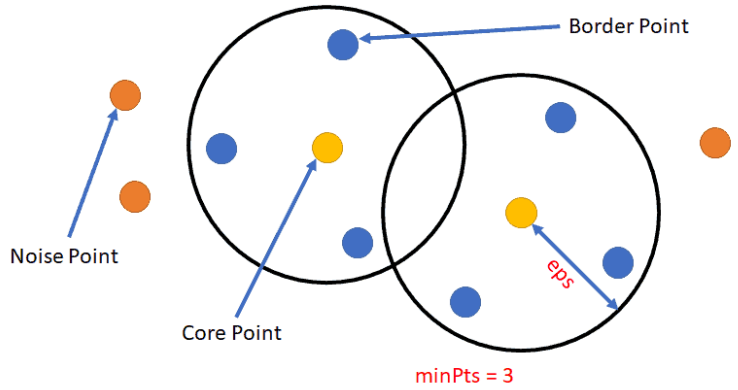
DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Key Parameters:

- **eps (ε)** : Maximum neighbor distance
- **MinPts** : Minimum points per cluster

Advantages:

- Arbitrary cluster shapes
- Robust to noise/outliers
- No predefined cluster count



Limitations:

- Struggles with clusters of varying density
- Poor in high-dimensional spaces (distance concentration)
- Border-point ambiguity
- Not hierarchical — use OPTICS/HDBSCAN for multi-scale
- Computationally expensive on large data

DBSCAN Variants and Extensions

Enhanced Versions:

- **ADBSCAN**: Adaptive parameter estimation
- **SS-DBSCAN**: Stratified sampling for ε estimation
- **STRP-DBSCAN**: Parallel processing for trajectory data

Applications:

- Crime hotspot detection
- Environmental monitoring
- Urban planning
- Transportation analysis

HDBSCAN — Hierarchical DBSCAN

What is HDBSCAN?

- HDBSCAN is a **hierarchical**, **density-based** clustering extension that builds a cluster hierarchy from DBSCAN-like density estimates and extracts the most stable clusters.
- **Key ideas:** **no fixed eps**, clusters persist across scales, stability score for clusters.
- Good when data have **variable-density** clusters and when tuning a single ε is hard.

Key parameters

- **min_cluster_size** — minimum size of any cluster (controls granularity)
- **min_samples** — (optional) controls how conservative the algorithm is (affects core distances)
- **metric** — same idea as DBSCAN (use **haversine** for lat/lon)
- sklearn-native: **from sklearn.cluster import HDBSCAN**

HDBSCAN

Advantages

- Automatically finds clusters at multiple density levels.
- Robust to variable-density clusters and noise.
- Provides soft membership through cluster probability / stability.

Tradeoffs

- More computationally intensive than plain DBSCAN.
- Still needs thoughtful choice of **`min_cluster_size`** / **`min_samples`**.
- Interpreting hierarchical output requires inspecting cluster stability.

OPTICS — Ordering Points To Identify the Clustering Structure

OPTICS generalizes DBSCAN: instead of committing to a single radius, it sweeps a range of densities and records the order in which points become reachable. Reading the resulting reachability plot lets us pick cluster scales after the fact, which is exactly what we want when density varies across the map.

What is OPTICS?

- OPTICS creates an ordering of points based on density (reachability plot) instead of a single fixed ε.
- Good at revealing clustering structure across many density levels and finding clusters of varying density.
- Produces: **reachability_**, **ordering_**, **core_distances_** — useful diagnostic plots.

Key parameters

- **min_samples** — minimum points in a neighborhood (controls cluster granularity)
- **max_eps** — maximum neighborhood radius (set to large value to explore all scales)
- **metric** — use **haversine** for lat/lon (fit on radians)
- sklearn-native: **from sklearn.cluster import OPTICS**

Advantages

- Reveals full density structure (**reachability plot**) — helps pick meaningful cluster scales.
- Handles **variable-density** data better than single-ε DBSCAN.
- Can be post-processed to extract flat clusters at chosen eps or using **xi**-style extraction.

Tradeoffs

- More effort to interpret.
- Computationally heavier than DBSCAN (but still scalable).
- Often paired with visual/manual inspection or automatic extraction rules.

Regionalization

What is Regionalization?

Spatial Contiguity Constraint

Unlike standard clustering, regionalization ensures that:

- All cluster members are **geographically connected**
- Forms single, unbroken areas
- No spatial fragmentation
- Practical for administrative boundaries

Applications:

- Electoral districts
- Sales territories
- Ecological zones
- Health service areas
- School catchment areas

SKATER Algorithm

SKATER turns the contiguity graph into a minimum spanning tree, then prunes the edges that join the most dissimilar neighbours. Each cut splits the tree into a connected sub-region, so the resulting clusters are guaranteed contiguous by construction.

SKATER (Spatial K'luster Analysis by Tree Edge Removal)

Algorithm Steps:

1. Build Minimum Spanning Tree (MST)
2. Remove edges to maximize dissimilarity
3. Creates spatially contiguous regions
4. Balances homogeneity and contiguity

Parameters:

- Number of clusters
- Floor constraint (minimum size)
- Attribute variables

Refs: Assunção et al. (2006); Rey & Anselin (2007)

```python
from libpysal.graph import Graph
from spopt.region import Skater

# modern contiguity Graph -> legacy W bridge
g = Graph.build_contiguity(gdf)   # Queen by default
w = g.to_W(); w.transform = "r"

# attrs_name MUST be a list of columns
attrs = ["median_house_value", "pct_rented"]

model = Skater(
    gdf, w, attrs_name=attrs,
    n_clusters=5,
    floor=3
)
model.solve()

gdf["clusters"] = model.labels_
```

Max-P Regions

Where SKATER fixes the *number* of regions, Max-P fixes a *minimum size* and finds the **largest** number of contiguous regions that clear it — the count is an output, not an input.

Max-P: Maximum number of regions with constraints

Objective:

- Maximize number of regions
- Subject to threshold constraints
- Maintain spatial contiguity
- Minimize within-region heterogeneity

Example Use Cases:

- Population-based districts
- Service coverage areas
- Resource allocation zones

Refs: Duque et al. (2012); Laura & Duque (2017)

Python Implementation:

```python
from spopt.region import MaxPHeuristic

# reuse the same g, w bridge as SKATER;
# attrs is a list of columns
attrs = ["median_house_value", "pct_rented"]

model = MaxPHeuristic(
    gdf, w, attrs,
    threshold_name="total_pop",
    threshold=50000
)
model.solve()

print(f"Found {model.p} regions")
gdf["maxp_labels"] = model.labels_
```

Worked Example — Regionalizing San Diego

We take 628 census tracts (`sandiego_tracts.gpkg`, EPSG:3857), build a Queen-contiguity `Graph`, bridge it to a row-standardized `W`, and run both a fixed-`k` SKATER and a population-constrained Max-P on the same inputs.

```python
import geopandas as gpd
from libpysal.graph import Graph
from spopt.region import Skater, MaxPHeuristic

gdf = gpd.read_file("data/sandiego/sandiego_tracts.gpkg")
attrs = ["median_house_value", "pct_rented",
         "median_hh_income"]          # list!

g = Graph.build_contiguity(gdf)       # modern API
w = g.to_W(); w.transform = "r"       # spopt bridge
```

```python
# fixed number of contiguous regions
sk = Skater(gdf, w, attrs_name=attrs, n_clusters=6)
sk.solve()
gdf["skater"] = sk.labels_            # -> 6 regions

# as many regions as a population floor allows
mp = MaxPHeuristic(gdf, w, attrs,
        threshold_name="total_pop", threshold=50000)
mp.solve()
print(mp.p)                           # -> 55 regions
```

> **Lesson —** same graph, two questions: SKATER returns the *k* you ask for; Max-P lets the **threshold** (≥ 50,000 residents) decide how many the data support — here 55.

> Notebook: `notebooks/07_clustering_and_regionalization.ipynb`

AZP — Automatic Zoning Procedure

AZP is a local-search regionalizer: it starts from a feasible partition and moves boundary units between neighbouring zones whenever that lowers within-region heterogeneity.

Objective

- Relocate spatial units to improve a global objective (within-region homogeneity).
- Start from an initial partition (random or seeded); move boundary units when the move reduces the objective.

Key ideas

- Minimise **within-region variance** (or another heterogeneity measure).
- Parameters: **p** (regions), **floor** (min region size), attributes driving homogeneity.
- Stochastic: multiple random starts to avoid poor local optima.

AZP — strengths & trade-offs

Advantages

- Flexible and fast for moderate-sized problems.
- Produces compact, contiguous regions for administrative or operational zones.
- Easy to tune for a target number of regions.

Trade-offs

- Local optima — use many starts.
- Sensitive to the initial seed and floor constraints.
- No global-optimum guarantee; evaluate and run several times.

> **Lesson —** AZP is a heuristic — run it from several random starts and keep the best; a single run can settle in a poor local optimum.

Hierarchical Spatial Clustering

Ward's agglomerative clustering becomes spatial once we feed it a connectivity matrix: only geographically adjacent units are allowed to merge. The sparse adjacency from the contiguity `Graph` supplies exactly that constraint, so every merge keeps the growing cluster connected.

Ward with Spatial Constraints

Traditional Approach:

- Standard linkage criteria
- No spatial consideration
- May create fragmented clusters

Spatially Constrained:

- Connectivity matrix from spatial weights
- Only adjacent units can merge
- Guarantees spatial contiguity

```
from sklearn.cluster import AgglomerativeClustering
from libpysal.graph import Graph

# contiguity Graph -> sparse connectivity
g = Graph.build_contiguity(gdf)

# only adjacent units may merge
model = AgglomerativeClustering(
    linkage="ward",
    connectivity=g.sparse,
    n_clusters=5
)
model.fit(scaled_data)
```

Mixed Approaches

ClustGeo: Geography + Attributes

- Combines geographical and feature space dissimilarity
- Weighted balance between spatial and attribute similarity
- Parameter α controls spatial constraint strength

GWR-Based Clustering:

- Uses Geographically Weighted Regression coefficients
- Identifies regions with similar local relationships
- Captures spatial heterogeneity in relationships

STICC (Spatial Toeplitz Inverse Covariance-Based Clustering):

- Markov random field approach
- Considers both attributes and spatial relationships
- Encourages spatial consistency

References: Chavent et al. (2018), Kang et al. (2022)

Tools

spopt - Spatial Optimization

- **SKATER**, **Max-P**, **AZP** algorithms
- Part of PySAL ecosystem
- Comprehensive regionalization tools

scikit-learn - Machine Learning

- **DBSCAN** with spatial metrics
- **AgglomerativeClustering** with connectivity
- Preprocessing and evaluation tools

Resources

Jupyter Notebooks:

1. **Geographic Data Science Book**
   - https://geographicdata.science/book/notebooks/10_clustering_and_regionalization.html
   - Complete tutorial with San Diego data
   - K-means, hierarchical, and regionalization
2. **PySAL Tutorials**
   - https://github.com/sjsrey/pysal-scipy22
   - SciPy conference materials
   - Hands-on geodemographic analysis
3. **spopt Documentation**
   - https://pysal.org/spopt/notebooks/skater.html
   - SKATER tutorial with Chicago Airbnb data

Key Parameters and Evaluation

Parameter Selection Guidelines:

DBSCAN:

- Use k-distance plots for ε
- Consider geographic scale
- Haversine metric for lat/lon

SKATER:

- Floor parameter for minimum size
- Attribute standardization crucial
- Balance homogeneity vs. contiguity

Max-P:

- Threshold drives number of regions
- Multiple runs for best solution
- Consider population distribution

Hierarchical:

- Linkage criterion matters
- Spatial weights construction
- Dendrogram interpretation

Evaluation Metrics

A good spatial clustering satisfies two families of criteria at once: regions must be geographically **coherent**, and members must be **similar in attribute space**.

Spatial coherence

- **Isoperimetric Quotient**: compactness
- **Join Count Statistics**: spatial contiguity
- **Spatial Autocorrelation**: Moran's I within clusters

Feature coherence

- **Calinski-Harabasz**: cluster separation
- **Silhouette**: assignment quality
- **Within-cluster SS**: homogeneity

Evaluation Metrics — comparing solutions

Once you have candidate clusterings, compare them against one another:

- **Adjusted Mutual Information**: agreement between two solutions
- **Rand Index**: pairwise clustering agreement
- **Modularity**: network-based quality

> **Lesson —** score every clustering on *both* axes — spatial coherence and feature homogeneity — never on one alone; the two can disagree.

Reading the Internal Validity Metrics

Two scores summarize how tight and well-separated the clusters are in feature space — read them together, and always against the spatial-coherence checks, never alone.

Silhouette — per-point fit, averaged over all points

$$s(i) = \frac{b(i) - a(i)}{\max\{\, a(i),\, b(i) \,\}} \in [-1, 1]$$

- $a(i)$: mean distance from i to points in its **own** cluster.
- $b(i)$: mean distance from i to the **nearest neighbouring** cluster.
- Read it: $\to +1$ well inside its cluster; $\to 0$ on a boundary; < 0 likely misassigned.

Calinski–Harabasz — between/within variance ratio

$$\mathrm{CH} = \frac{\mathrm{tr}(B_k)}{\mathrm{tr}(W_k)} \cdot \frac{n - k}{k - 1}$$

- B_k, W_k: between- and within-cluster scatter; n points, k clusters.
- Read it: **higher is better**; no fixed scale, so compare across candidate k and keep the peak.

> Both are geometry-only: a high score can still hide spatially fragmented regions, so pair them with Moran's I or join-count checks.

Applications — planning & policy

Regionalization surfaces wherever connected, homogeneous zones matter. The same handful of algorithms recurs across very different domains.

Urban Planning:

- Neighborhood delineation
- Service area optimization
- Zoning and land use planning

Public Health:

- Disease surveillance regions
- Healthcare catchment areas
- Epidemiological pattern detection

Political Geography:

- Electoral district design
- Gerrymandering analysis
- Voting pattern regionalization

Environmental Science:

- Ecological zone identification
- Climate region mapping
- Pollution monitoring networks

Applications — health, mobility & beyond

Spatial Transcriptomics:

- Gene expression clustering in tissue
- Spatial pattern discovery in biology
- Integration of location and molecular data

Transportation:

- Traffic flow regionalization
- Service territory optimization
- Mobility pattern analysis

Social Media Analysis:

- Geographic sentiment clustering
- Event detection and localization
- Social network spatial patterns

Refs: Biswas et al. (2020); Chen et al. (2023); Wang et al. (2024)

Summary

Spatial Clustering ≠ Standard Clustering

- Geographic relationships matter fundamentally
- Balances similarity and proximity
- Requires specialized algorithms and considerations

Method Selection Guide:

- **DBSCAN**: Irregular shapes, noise tolerance
- **SKATER**: Contiguous regions, balanced sizes
- **Max-P**: Maximize regions with constraints
- **Hierarchical**: Nested structures, dendrograms

Python Ecosystem Maturity:

- **spopt** for regionalization
- **scikit-learn** for density-based methods
- **PySAL** ecosystem for comprehensive spatial analysis

Best Practices

Three habits make a regionalization defensible: prepare the inputs, tune stochastic methods with multiple random starts, and validate with several metrics plus domain knowledge.

Data Preparation:

1. **Standardize variables** (critical for distance-based methods)
2. **Choose appropriate spatial weights** (Queen, Rook, k-NN)
3. **Handle missing data** carefully (affects spatial relationships)

Parameter Tuning:

1. **Cross-validation** with spatial considerations
2. **Multiple random starts** for stochastic algorithms
3. **Visual inspection** of results for geographic sensibility

Evaluation Strategy:

1. **Multiple metrics**: spatial + attribute coherence
2. **Domain knowledge validation**
3. **Sensitivity analysis** for key parameters

References

- Ester, M., et al. (1996). A density-based algorithm for discovering clusters. *KDD-96*.
- Ng, R. T., & Han, J. (1994). Efficient and effective clustering methods for spatial data mining. *VLDB*.
- Assunção, R. M., et al. (2006). Efficient regionalization techniques for socio-economic geographical units. *International Journal of Geographical Information Science*.
- Kang, Y., et al. (2022). STICC: A multivariate spatial clustering method. *arXiv:2203.09611*.
- Chavent, M., et al. (2018). ClustGeo: an R package for hierarchical clustering with spatial constraints. *Computational Statistics*.